

TimeStamp Attribute in Entity Framework Core (EF Core)

In this article, I will discuss **TimeStamp Data Annotation Attribute in Entity Framework Core (EF Core)** with Examples. Please read our previous article discussing [DatabaseGenerated Attribute in Entity Framework Core](#) with Examples.

In Entity Framework Core (EF Core), managing concurrent operations on data is essential, especially in multi-user environments where multiple users or transactions might attempt to update or delete the same data simultaneously. Concurrency tokens are a key concept that helps detect and resolve such conflicts. One way to implement **Concurrency Tokens** in EF Core is using the [Timestamp] attribute.

What is Timestamp Attribute in Entity Framework Core?

In Entity Framework Core (EF Core), the Timestamp attribute specifies that a particular **Byte Array Property** should be treated as a concurrency token. A concurrency token ensures that the data being updated or deleted has not changed since it was last read, providing a way to manage concurrent operations on the data.

When we apply the Timestamp attribute to a property in our entity class, the corresponding column in the database is treated as a Row Version column. SQL Server automatically updates this column value every time a row is inserted or modified. This auto-updating mechanism lets EF Core detect if concurrent changes have occurred since the data was last fetched.

EF Core will include a Row Version column in the WHERE clause of UPDATE and DELETE SQL statements to ensure that the record hasn't changed since it was last fetched. If the record has been modified, the operation will fail, allowing us to handle the concurrency conflict appropriately.

Definition of Timestamp Attribute in EF Core

If you go to the definition of TimeStamp Attribute in Entity Framework Core, you will see the following. This class has a parameter-less constructor.

```
namespace System.ComponentModel.DataAnnotations
{
    ... public sealed class TimestampAttribute : Attribute
    {
        ... public TimestampAttribute();
    }
}
```

Key Features of the Timestamp Attribute in EF Core:

- The Timestamp attribute can only be applied once per entity and must be applied to a byte[] (byte array) property.
- EF Core maps this property to the TimeStamp data type in SQL Server and uses it automatically for concurrency checks in UPDATE and DELETE SQL statements.
- The Timestamp attribute cannot be applied to properties of other data types. It is exclusive to byte[] arrays.

Real-Time Example of a Concurrency Issue Without Using the TimeStamp Attribute

Let's consider an example of a simple e-commerce application. Let us first create the Product entity. So, create a class file named **Product.cs** within the **Entities** folder and then copy and paste the following code.

```
using System.ComponentModel.DataAnnotations.Schema;
namespace EFCoreCodeFirstDemo.Entities
{
    public class Product
    {
        public int ProductId { get; set; }
        public string Name { get; set; }
        [Column(TypeName = "decimal(10,2)")]
        public decimal Price { get; set; }
        public int StockQuantity { get; set; }
    }
}
```

Setting Up the DbContext:

Next, modify the EFCoreDbContext class as follows.

```
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-6P5NK25R\SQLSERVER2022DEV;Database=ECommerceDB;Trusted_Connection=True;TrustServerCertificate=True");
        }
        //Overriding the OnModelCreating method to add seed data
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            // Seeding Product data
            modelBuilder.Entity<Product>().HasData(
                new Product { ProductId = 1, Name = "Laptop", Price = 5000, StockQuantity = 10 },
                new Product { ProductId = 2, Name = "Desktop", Price = 3000, StockQuantity = 20 },
                new Product { ProductId = 3, Name = "Mobile", Price = 1500, StockQuantity = 50 }
            );
        }
        public DbSet<Product> Products { get; set; }
    }
}
```

Generating and Applying Migration:

Open Package Manager Console and Execute **Add-Migration** and **Update-Database** commands as follows.

```
Package Manager Console
Package source: All [v] [g] Default project: EFCoreCodeFirstDemo [v]
PM> Add-Migration EFCoreDBMig1
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM> Update-Database
Build started...
Build succeeded.
Applying migration '20240914070320_EFCoreDBMig1'.
Done.
```

Once the above commands are executed, verify the database. The ECommerceDB with the Products table should have been created, as shown in the image below.

ECommerceDB

- Database Diagrams
- Tables
 - System Tables
 - FileTables
 - External Tables
 - Graph Tables
 - dbo.__EFMigrationsHistory
 - dbo.Products
 - Columns
 - ProductId (PK, int, not null)
 - Name (nvarchar(max), not null)
 - Price (decimal(10,2), not null)
 - StockQuantity (int, not null)

Now, if you verify the Products table, then you will see the following data:

SELECT * FROM Products

Results

Messages

ProductId	Name	Price	StockQuantity
1	Laptop	5000.00	10
2	Desktop	3000.00	15
3	Mobile	1500.00	20

Real-Time Example to Understand Concurrency Issue:

Suppose two users are trying to place an order with the same product simultaneously. The following is the Ideal Scenario:

- User A fetches the product data (Laptop with Product ID 1) from the database and sees that the stock is 10.
- User B also fetches the same product data and sees that the stock is 10.
- User A books three units of the product and updates the stock to 7 in the database.
- User B, unaware of User A's booking, also books seven units and updates the stock to 3 in the database.

Now, both updates succeed, and the final stock becomes 3. However, this is incorrect because the product's stock should have been 0. For a better understanding, please modify the Program class as follows. The following code is self-explained, so please read the comment lines for a better understanding.

```
using EFCoreCodeFirstDemo.Entities;
namespace EFCoreCodeFirstDemo
{
    public class Program
    {
        // Define the quantities each user is booking
        private static int userAQuantity = 3;
        private static int userBQuantity = 7;
        static void Main(string[] args)
        {
            try
            {
                //This Propetry will store the Initia Stock
                int InitialStock = 0;
                // Get initial stock from database before any booking
                using (var context = new EFCoreDbContext())
                {
                    var initialProduct = context.Products.Find(1);
                    InitialStock = initialProduct?.StockQuantity ?? 0;
                    Console.WriteLine($"Initial stock in database:
{initialProduct?.StockQuantity}");
                }
                // Create two threads to simulate concurrent transactions
                Thread t1 = new Thread(Method1); // Thread for User A's
transaction
                Thread t2 = new Thread(Method2); // Thread for User B's
transaction

                Console.WriteLine("Booking Started by User A and User B");
                // Start both threads for User A and User B
                t1.Start();
                t2.Start();
                // Ensure both threads finish before proceeding
                t1.Join();
                t2.Join();
                // After the transactions get the final stock in the database
                using (var context = new EFCoreDbContext())
                {
```

```

        var finalProduct = context.Products.Find(1); // Fetch
product with Id 1
        int expectedFinalStock = InitialStock - (userAQuantity +
userBQuantity); // Calculate the expected final stock
        Console.WriteLine($"Expected final stock (Initial stock -
User A booking - User B booking): {expectedFinalStock}");
        Console.WriteLine($"Final stock in database:
{finalProduct?.StockQuantity}");
        if (finalProduct != null && finalProduct.StockQuantity ==
expectedFinalStock)
        {
            Console.WriteLine("Final stock is correct.");
        }
        else
        {
            Console.WriteLine("Final stock is incorrect. There
may be a concurrency issue.");
        }
    }
    Console.ReadKey(); // Wait for user to press a key before
exiting
}
catch (Exception ex)
{
    Console.WriteLine($"Error: {ex.Message}"); // Catch and log
any errors
}
}
// User A | Transaction 1 | Booking 3 items
public static void Method1()
{
    using EFCoreDbContext context = new EFCoreDbContext(); // Create
a new DB context
    // Fetch product with Id 1
    var product1 = context.Products.Find(1);
    // Simulate delay (2 seconds) to mimic real-world concurrency
issues
    Thread.Sleep(TimeSpan.FromSeconds(2));
    if (product1 != null) // Ensure product exists
    {
        // Update stock quantity for User A's booking
        product1.StockQuantity -= userAQuantity;
        // Save changes to the database
        context.SaveChanges();
        // Log how many quantities User A booked and the new stock
quantity
        Console.WriteLine($"User A booked {userAQuantity} items.
Updated stock after User A: {product1.StockQuantity}");
    }
}
// User B | Transaction 2 | Booking 7 items

```

```

    public static void Method2()
    {
        using EFCoreDbContext context = new EFCoreDbContext(); // Create
a new DB context
        // Fetch product with Id 1
        var product1 = context.Products.Find(1);
        // Simulate delay (5 seconds) to allow time for potential
concurrency conflict
        Thread.Sleep(TimeSpan.FromSeconds(5));
        if (product1 != null) // Ensure product exists
        {
            // Update stock quantity for User B's booking
            product1.StockQuantity -= userBQuantity;
            // Save changes to the database
            context.SaveChanges();
            // Log how many quantities User B booked and the new stock
quantity
            Console.WriteLine($"User B booked {userBQuantity} items.
Updated stock after User B: {product1.StockQuantity}");
        }
    }
}

```

Output:

```

Initial stock in database: 10
Booking Started by User A and User B
User A booked 3 items. Updated stock after User A: 7
User B booked 7 items. Updated stock after User B: 3
Expected final stock (Initial stock - User A booking - User B booking): 0
Final stock in database: 3
Final stock is incorrect. There may be a concurrency issue.

```

Why Are We Facing the Concurrency Issue?

The issue arises because both users read the same data and then overwrite each other's changes. The database has no mechanism to detect that the data was modified between the time it was read and when it was written back. This is known as the Concurrency Problem.

How Can the Above Issue Be Overcome Using the TimeStamp Attribute?

We can introduce a TimeStamp property in the Product entity to handle the concurrency issue with Entity Framework Core. EF Core will throw a concurrency exception if it detects that the data has changed between loading it and attempting an update. Let's see how to implement this in our example.

So, first, modify the Product entity as follows to include the TimeStamp attribute on a byte array type property. Here, you can see we have created one property named RowVersion (you can give any name) and decorated that property with the TimeStamp Attribute.

```

using System.ComponentModel.DataAnnotations;

```

```

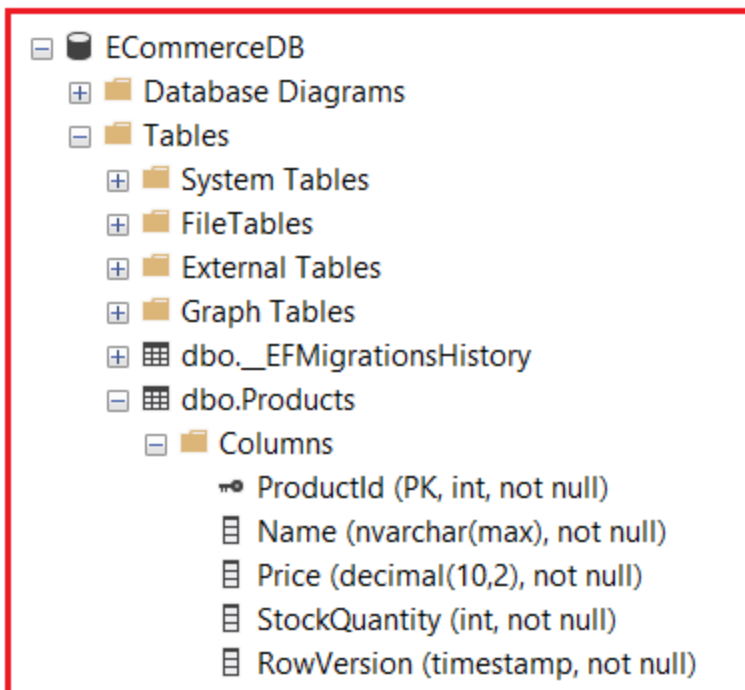
using System.ComponentModel.DataAnnotations.Schema;
namespace EFCoreCodeFirstDemo.Entities
{
    public class Product
    {
        public int ProductId { get; set; }
        public string Name { get; set; }
        [Column(TypeName = "decimal(10,2)")]
        public decimal Price { get; set; }
        public int StockQuantity { get; set; }
        //One Table Can have only one TimeStamp Column
        //It should be applied on a byte[] property
        [Timestamp]
        public byte[] RowVersion { get; set; }
    }
}

```

Note: Before proceeding further, delete your project's existing database and Migration folder.

Again, Generating and Applying Migration:

Then, open the Package Manager Console and Execute the **Add-Migration** and **Update-Database** commands. Once you execute the commands, verify the database and this time, it should include the RowVersion column with TimeStamp data type as shown in the below image:



Now, if you verify the database table, you should see the RowVersion is automatically filled with some unique values, as shown in the below image:

```
SELECT * FROM Products
```

ProductId	Name	Price	StockQuantity	RowVersion
1	Laptop	5000.00	10	0x000000000000007D1
2	Desktop	3000.00	15	0x000000000000007D2
3	Mobile	1500.00	20	0x000000000000007D3

Note: The Row Version value is automatically inserted when we insert a new entity and updated automatically every time we modify the Entity data. EF Core will also use this Column value in the WHERE Clause when performing the UPDATE and DELETE Operations.

Modifying the Context Class:

We need to see what SQL Statement is generated by EF Core when performing the UPDATE Operations, and also, we need to see whether the RowVersion column is used in the WHERE Clause of the UPDATE SQL Statement. To see this, we need to enable EF Core Logging. So, modify the EFCoreDbContext class as follows:

```
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Logging;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            // To Display the Generated the Database Script
            optionsBuilder.LogTo(Console.WriteLine, LogLevel.Information);
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-6P5NK25R\SQLSERVER2022DEV;Database=ECommerceDB;Trusted_Connection=True;TrustServerCertificate=True");
        }
        //Overriding the OnModelCreating method to add seed data
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            // Seeding Product data
            modelBuilder.Entity<Product>().HasData(
                new Product { ProductId = 1, Name = "Laptop", Price = 5000, StockQuantity = 10 },
                new Product { ProductId = 2, Name = "Desktop", Price = 3000, StockQuantity = 15 },
                new Product { ProductId = 3, Name = "Mobile", Price = 1500, StockQuantity = 20 }
            );
        }
        public DbSet<Product> Products { get; set; }
    }
}
```

Modifying the Program Class:

Next, modify the Program class as follows. The code is the previous example where we performed the update operation with the same product by two different users. But, this time, one of the Update operations will be Failed.

```
using EFCoreCodeFirstDemo.Entities;
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo
{
    internal class Program
    {
        // Define the quantities each user is booking
        private static int userAQuantity = 3;
        private static int userBQuantity = 7;
        // Flags to track if the operations were successful
        private static bool userAOperationSuccess = false;
        private static bool userBOperationSuccess = false;
        static void Main(string[] args)
        {
            try
            {
                // This property will store the initial stock
                int initialStock = 0;
                // Get initial stock from the database before any booking
                using (var context = new EFCoreDbContext())
                {
                    var initialProduct = context.Products.Find(1);
                    initialStock = initialProduct?.StockQuantity ?? 0;
                    Console.WriteLine($"Initial stock in database:
{initialStock}");
                }
                // Create two threads to simulate concurrent transactions
                Thread t1 = new Thread(Method1); // Thread for User A's
transaction
                Thread t2 = new Thread(Method2); // Thread for User B's
transaction

                Console.WriteLine("Booking started by User A and User B");
                // Start both threads for User A and User B
                t1.Start();
                t2.Start();
                // Ensure both threads finish before proceeding
                t1.Join();
                t2.Join();
                // Get the final stock in the database
                using (var context = new EFCoreDbContext())
                {
                    var finalProduct = context.Products.Find(1); // Fetch
product with Id 1
                    // Calculate the expected final stock based on successful
operations
```

```

        int expectedFinalStock = initialStock;
        if (userAOperationSuccess)
        {
            expectedFinalStock = expectedFinalStock -
userAQuantity;
        }
        if (userBOperationSuccess)
        {
            expectedFinalStock = expectedFinalStock -
userBQuantity;
        }
        Console.WriteLine($"Expected Final Stock:
{expectedFinalStock}");
        Console.WriteLine($"Final Stock in Database:
{finalProduct?.StockQuantity}");
        if (finalProduct != null && finalProduct.StockQuantity ==
expectedFinalStock)
        {
            Console.WriteLine("Final Stock is Correct.");
        }
        else
        {
            Console.WriteLine("Final Stock is Incorrect. There
may be a concurrency issue.");
        }
    }
    Console.ReadKey(); // Wait for the user to press a key before
exiting
}
catch (Exception ex)
{
    Console.WriteLine($"Error: {ex.Message}"); // Catch and log
any errors
}
}
// User A | Transaction 1 | Booking 3 items
public static void Method1()
{
    try
    {
        using EFCoreDbContext context = new EFCoreDbContext(); //
Create a new DB context
        // Fetch product with Id 1
        var product1 = context.Products.Find(1);
        // Simulate delay (2 seconds) to mimic real-world concurrency
issues
        Thread.Sleep(TimeSpan.FromSeconds(2));
        if (product1 != null) // Ensure product exists
        {
            // Update stock quantity for User A's booking
            product1.StockQuantity -= userAQuantity;

```

```

        // Save changes to the database
        context.SaveChanges();
        // Log success and set the operation success flag
        Console.WriteLine($"User A booked {userAQuantity} items.
Updated stock after User A: {product1.StockQuantity}");
        userAOperationSuccess = true;
    }
}
catch (DbUpdateException ex)
{
    // Log error and set the operation success flag to false
    Console.WriteLine($"User A Error: {ex.InnerException?.Message
?? ex.Message}");
    userAOperationSuccess = false;
}
}
// User B | Transaction 2 | Booking 7 items
public static void Method2()
{
    try
    {
        using EFCoreDbContext context = new EFCoreDbContext(); //
Create a new DB context
        // Fetch product with Id 1
        var product1 = context.Products.Find(1);
        // Simulate delay (5 seconds) to allow time for potential
concurrency conflict
        Thread.Sleep(TimeSpan.FromSeconds(5));
        if (product1 != null) // Ensure product exists
        {
            // Update stock quantity for User B's booking
            product1.StockQuantity -= userBQuantity;
            // Save changes to the database
            context.SaveChanges();
            // Log success and set the operation success flag
            Console.WriteLine($"User B booked {userBQuantity} items.
Updated stock after User B: {product1.StockQuantity}");
            userBOperationSuccess = true;
        }
    }
    catch (Exception ex)
    {
        // Log error and set the operation success flag to false
        Console.WriteLine($"User B Error: {ex.InnerException?.Message
?? ex.Message}");
        userBOperationSuccess = false;
    }
}
}
}
}

```

Output:

Let us assume Method 1 starts updating first. He will update the data in the database and the RowVersion value. Now, Method 2 tries to update the same entity. If you remember, it will use the RowVersion column while updating, but that RowVersion value with the Method2 entity has already been modified by Method1. So, Method2 has a RowVersion value that no longer exists in the database; hence, the Method2 SaveChanges method will throw an exception showing concurrency issues.

```
info: 18-11-2024 09:33:51.287 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
      Executed DbCommand (35ms) [Parameters=[@__p_0=?' (DbType = Int32)], CommandType='Text', CommandTimeout='30']
      SELECT TOP(1) [p].[ProductId], [p].[Name], [p].[Price], [p].[RowVersion], [p].[StockQuantity]
      FROM [Products] AS [p]
      WHERE [p].[ProductId] = @__p_0
Initial stock in database: 10
Booking started by User A and User B
info: 18-11-2024 09:33:51.255 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
      Executed DbCommand (1ms) [Parameters=[@__p_0=?' (DbType = Int32)], CommandType='Text', CommandTimeout='30']
      SELECT TOP(1) [p].[ProductId], [p].[Name], [p].[Price], [p].[RowVersion], [p].[StockQuantity]
      FROM [Products] AS [p]
      WHERE [p].[ProductId] = @__p_0
info: 18-11-2024 09:33:51.259 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
      Executed DbCommand (1ms) [Parameters=[@__p_0=?' (DbType = Int32)], CommandType='Text', CommandTimeout='30']
      SELECT TOP(1) [p].[ProductId], [p].[Name], [p].[Price], [p].[RowVersion], [p].[StockQuantity]
      FROM [Products] AS [p]
      WHERE [p].[ProductId] = @__p_0
info: 18-11-2024 09:33:53.299 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
      Executed DbCommand (9ms) [Parameters=[@p1=?' (DbType = Int32), @p2=?' (Size = 8) (DbType = Binary), @p0=?' (DbType = Int32)], CommandType='Text', CommandTimeout='30']
      SET IMPLICIT_TRANSACTIONS OFF;
      SET NOCOUNT ON;
      UPDATE [Products] SET [StockQuantity] = @p0
      OUTPUT INSERTED.[RowVersion]
      WHERE [ProductId] = @p1 AND [RowVersion] = @p2;
User A booked 3 items. Updated stock after User A: 7
info: 18-11-2024 09:33:56.267 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
      Executed DbCommand (1ms) [Parameters=[@p1=?' (DbType = Int32), @p2=?' (Size = 8) (DbType = Binary), @p0=?' (DbType = Int32)], CommandType='Text', CommandTimeout='30']
      SET IMPLICIT_TRANSACTIONS OFF;
      SET NOCOUNT ON;
      UPDATE [Products] SET [StockQuantity] = @p0
      OUTPUT INSERTED.[RowVersion]
      WHERE [ProductId] = @p1 AND [RowVersion] = @p2;
User B Error: The database operation was expected to affect 1 row(s), but actually affected 0 row(s); data may have been modified or deleted since entities were loaded.
nkId=527962 for information on understanding and handling optimistic concurrency exceptions.
info: 18-11-2024 09:33:56.337 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
      Executed DbCommand (1ms) [Parameters=[@__p_0=?' (DbType = Int32)], CommandType='Text', CommandTimeout='30']
      SELECT TOP(1) [p].[ProductId], [p].[Name], [p].[Price], [p].[RowVersion], [p].[StockQuantity]
      FROM [Products] AS [p]
      WHERE [p].[ProductId] = @__p_0
Expected Final Stock: 7
Final Stock in Database: 7
Final Stock is Correct.
```

How EF Core Handling Concurrency Issues with TimeStamp Attribute:

In the above example, every time the product is updated, the RowVersion is also updated automatically in the database. When EF Core tries to update the product's stock, it includes the RowVersion in the WHERE clause of the SQL UPDATE statement:

```
UPDATE Products
SET Stock = @Stock,
WHERE Id = @Id AND RowVersion = @CurrentRowVersion;
```

If the RowVersion in the database doesn't match the RowVersion sent by EF Core (meaning another user has updated the row), the UPDATE statement will fail, and EF Core will throw a `DbUpdateConcurrencyException`. This exception informs us that a concurrency conflict occurred, and we can take appropriate actions, such as:

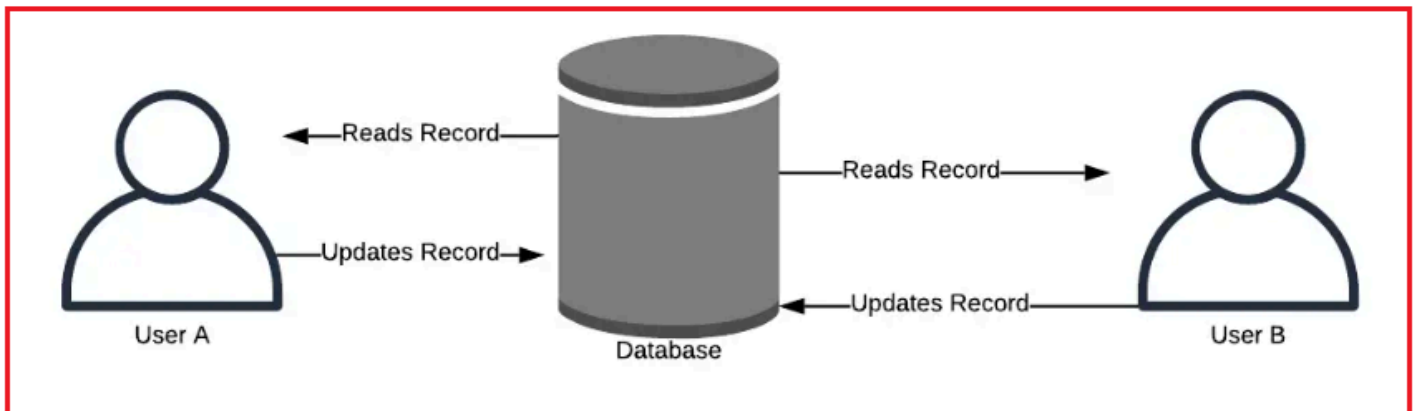
- Prompting the user to reload the data.
- Automatically retrying the update.

Now, if you verify the database table, then you will also see that the RowVersion of the ProductId is updated, as shown in the below image:

```
SELECT * FROM Products
```

ProductId	Name	Price	StockQuantity	RowVersion
1	Laptop	5000.00	7	0x000000000000007D4
2	Desktop	3000.00	15	0x000000000000007D2
3	Mobile	1500.00	20	0x000000000000007D3

For a better understanding, please have a look at the following diagram.



With our example, we can understand a concurrency scenario: User A and User B interact with the same database record simultaneously.

Initial Read by Both Users:

- User A reads a specific record from the database.
- User B also reads the same record at the same time.
- At this point, both users have the same version of the record in memory, including the current value of the RowVersion (or Timestamp), which tracks changes to that record. The database still holds the original version of the record, and both users have identical data in their application's memory.

User A: Updates the Record:

After reading the record, User A made some changes and submitted an update request to the database. When User A's changes are saved, the RowVersion column is included in the WHERE clause of the UPDATE statement in SQL.

This ensures that the update only occurs if the RowVersion in the database matches the value User A originally read. Since no other updates have been made, the RowVersion matches, and the record has been successfully updated. The RowVersion in the database has now been automatically incremented to reflect this change.

User B: Attempts to Update the Same Record:

User B also makes some changes and tries to update the same record, unaware that User A has already modified it. When User B's UPDATE request is sent to the database, EF Core again includes the RowVersion in the WHERE clause to ensure no changes were made since User B last read the record. However, the RowVersion in the database has now been updated by User A, so it no longer matches the value User B originally read.

Concurrency Conflict:

Since the RowVersion values don't match, EF Core detects that a concurrency conflict has occurred. In this case, EF Core throws a DbUpdateConcurrencyException, indicating that another user (User A) has modified the record after User B initially read it.

Why Do We Need the TimeStamp Attribute in Entity Framework Core?

In multi-user environments, such as a web application, several users can access and attempt to modify the same entity simultaneously. Without concurrency control, the last write operation will overwrite previous ones, leading to data inconsistency. The TimeStamp attribute helps us avoid such scenarios by preventing updates if the data has changed since it was last read. The TimeStamp attribute helps:

- **Prevent Data Overwrites:** Ensures a user doesn't unknowingly overwrite another user's changes.
- **Maintain Data Integrity:** Keeps the data consistent and reliable.
- **Handle Concurrency:** Allows applications to inform users about conflicts and take appropriate actions.

In the next article, I will discuss [ConcurrencyCheck Attribute in Entity Framework Core](#) with Examples. In this article, I try to explain the **TimeStamp Data Annotation Attribute in Entity Framework Core** with Examples. I hope you enjoyed this TimeStamp Attribute in EF Core with Examples article.